Machine Translated by Google

Orange Wireless Glasses SDK User Guide

Update log:

1. Introduction

1.1 SDK Function

2. API Description

2.0 Access Conditions

2.1 SDK Required Permissions

2.2 Access Conditions

2.3.1 API

Scanning Devices

Device connection: BleOperateManager.getInstance()

Device connection: BleOperateManager.getInstance()

2.3.2 Function List:

Synchronize time

Glasses battery life

Read glasses version information

Control glasses to take photos

Control glasses to record video

Control glasses to record audio

Control intelligent image recognition and report thumbnails

BT pairing

Glasses volume control

The amount of media not synced with the glasses

Glasses WIFI connection, photo and video guide, audio recording

SDK

1. Author: James

2. Shenzhen QC.wireless Technology Co., Ltd.

3. Version: 1.0.0

Machine Translated by Google

Update log:

1. (2025/07/23) Scan, connect, and measure instructions
2. (2025/07/23) Added setting instructions
3. (2025/08/08) Added WIFI related operations
4. (2025/08/16) Added glasses PCM data

1.

1.1 SDK Function

Provides partner companies with an Android Glass SDK that can be used with Orange Wireless devices, providing basic and advanced functionality for primary glasses or other devices. This document is intended to explain the API's usage context, functionality, etc. Expected Audience and Reading Recommendations The intended audience and reader recommendations for this document are shown in Appendix 1.

Reader	Effect
Software Architecture Engineer	Architecture analysis and technical guidance
Android development engineers have certain Android development capabilities and understand Ble and WiFi related development technologies	

2. API

2.0ÿÿ

> Android 5.0 or above, Bluetooth 4.0 or above.

2.1 SDK

Machine Translated by Google

```
//Network permissions
<uses-permission android:name="android.permission.INTERNET" />
//Bluetooth related permissions
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"
/>
//Storage related permissions
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission
android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

2.2ÿÿ

- Orange wireless glasses
- Orange Wireless SDK and documentation

2.3.1 API

Scanning Devices

```
//start
BleScannerHelper.getInstance().scanDevice(final Context context,
UUID mUuid, final ScanWrapperCallback scanCallBack);
//Finish
BleScannerHelper.getInstance().stopScan(Context context)
//Specify device scan
BleScannerHelper.getInstance().scanTheDevice(final Context context,
final String macAddress, final OnTheScanResult scanResult)
```

Machine Translated by Google

Device connection: BleOperateManager.getInstance()

```
//Direct connection

BleOperateManager.getInstance().connectDirectly(smartWatch.deviceAddres
s)

//Scan for connections

BleOperateManager.getInstance().connectWithScan(smartWatch.deviceAddres
s)

//disconnect
BleOperateManager.getInstance().unBindDevice()
//Reconnect
BleOperateManager.getInstance().setNeedConnect(boolean needConnect)
//Called when Bluetooth is turned off
BleOperateManager.getInstance().setBluetoothTurnOff(false)
BleOperateManager.getInstance().disconnect()
//Open system Bluetooth monitoring
BleOperateManager.getInstance().setBluetoothTurnOff(true)

Please note that the MyApplication sample has a registered listener. You must register first to receive Bluetooth connection and disconnection events.
```

Device connection: BleOperateManager.getInstance()

```
//Register glasses event reporting monitor
LargeDataHandler.getInstance().addOutDeviceListener(100,
deviceNotifyListener)

inner class MyDeviceNotifyListener : GlassesDeviceNotifyListener() {

@RequiresApi(Build.VERSION_CODES.O)
override fun parseData(cmdType: Int, response:
GlassesDeviceNotifyRsp) {
when (response.loadData[6].toInt()) {
//Glasses battery report
```

Machine Translated by Google

```
                0x05 -> {
                    //Current battery level
                    val battery = response.loadData[7].toInt()
                    //Is it charging?
                    val changing = response.loadData[8].toInt()
                }
                //Glasses are identified through quick recognition
                0x02 -> {
                    if (response.loadData.size > 9 &&
response.loadData[9].toInt() == 0x02) {
                        //To set recognition intent: eg Please help me see what is in front of me, the content in the picture
                    }
                    //Get the image thumbnail
                    LargeDataHandler.getInstance().getPictureThumbnails
{ cmdType, success, data ->
                        //Please save the data to the path, jpg image
                    }
                }

                0x03 -> {
                    if (response.loadData[7].toInt() == 1) {
                        // Turn on the microphone and start talking
                    }
                }
                //ota upgrade
                0x04 -> {
                    try {
                        val download = response.loadData[7].toInt()
                        val soc = response.loadData[8].toInt()
                        val nor = response.loadData[9].toInt()
                        //download firmware download progress soc download progress nor upgrade progress
                    } catch (e: Exception) {
                        e.printStackTrace()
                    }
                }
```

Machine Translated by Google

```
0x0c -> {
    //Glasses trigger pause event, voice broadcast
    if (response.loadData[7].toInt() == 1) {
        //to do
    }
}

0x0d -> {
    //Unbind APP event
    if (response.loadData[7].toInt() == 1) {
        //to do
    }
}
//Insufficient glasses memory event
0x0e -> {

}
//Translation pause event
0x10 -> {

}
//Glasses volume change event
0x12 -> {
    //Music volume
    //Minimum volume
    response.loadData[8].toInt()
    //Maximum volume
    response.loadData[9].toInt()
    //Current volume
    response.loadData[10].toInt()

    //Incoming call volume
    //Minimum volume
    response.loadData[12].toInt()
    //Maximum volume
    response.loadData[13].toInt()
```

Machine Translated by Google

```
                    //Current volume
                    response.loadData[14].toInt()

                    //Glasses system volume
                    //Minimum volume
                    response.loadData[16].toInt()
                    //Maximum volume
                    response.loadData[17].toInt()
                    //Current volume
                    response.loadData[18].toInt()

                    //Current volume mode
                    response.loadData[19].toInt()

                }
            }
        }
    }
```

:ÿÿ2.3.2

Synchronize time

```
LargeDataHandler.getInstance().syncTime {        _, _    -> }
```

Glasses battery life

Machine Translated by Google

```
        //Add power monitoring
        LargeDataHandler.getInstance().addBatteryCallBack("init") {                    _,
    response ->

        }
        //Balance
        LargeDataHandler.getInstance().syncBattery()
    }
```

Read glasses version information

```
        LargeDataHandler.getInstance().syncDeviceInfo { _, response ->
                        if (response != null) {
                            //wifi firmware version
                              response.wifiFirmwareVersion
                            //wifi product version
                            response.wifiHardwareVersion
                            //Bluetooth product version
                              response.hardwareVersion
                            //Bluetooth firmware version
                              response.firmwareVersion
                        }
                    }
    }
```

Control glasses to take photos

```
        LargeDataHandler.getInstance().glassesControl(
                        byteArrayOf(0x02, 0x01, 0x01)
                    ) {     _,    it ->
                        if (it.dataType == 1 && it.errorCode == 0) {
                            when (it.workTypeIng) {
                                2 -> {
                                        //The glasses are recording video
                                }
```

Machine Translated by Google

```
                    4 -> {
                        //The glasses are in transmission mode
                    }
                    5 -> {
                        //The glasses are in OTA mode
                    }
                    1, 6 ->{
                        //The glasses are in photo mode
                    }
                    7 -> {
                        //The glasses are communicating with AI
                    }
                    8 ->{
                        //The glasses are in recording mode
                    }
                }
            } else {
                //Start and end execution
            }
        }
```

Control glasses to record video

```
//videoStart true starts recording false stops recording
        val videoStart=true
        val value = if (videoStart) 0x02 else 0x03
        LargeDataHandler.getInstance().glassesControl(
            byteArrayOf(0x02, 0x01, value.toByte())
        ) {   _,   it ->
            if (it.dataType == 1) {
                if (it.errorCode == 0) {
                    when (it.workTypeIng) {
                        2 -> {
                            //The glasses are recording video
                        }
                        4 -> {
```

Machine Translated by Google

```
                    //The glasses are in transmission mode
                }
                5 -> {
                    //The glasses are in OTA mode
                }
                1, 6 ->{
                    //The glasses are in photo mode
                }
                7 -> {
                    //The glasses are communicating with AI
                }
                8 ->{
                    //The glasses are in recording mode
                }
            }
        } else {
            //Start and end execution
        }
    }
}
```

Control glasses to record audio

```
//recordStart true starts recording false stops recording
val recordStart=true
val value = if (recordStart) 0x08 else 0x0c
LargeDataHandler.getInstance().glassesControl(
        byteArrayOf(0x02, 0x01, value.toByte())
) {   _,   it ->
    if (it.dataType == 1) {
        if (it.errorCode == 0) {
            when (it.workTypeIng) {
                2 -> {
                    //The glasses are recording video
                }
                4 -> {
```

Machine Translated by Google

```
                    //The glasses are in transmission mode
                }
                5 -> {
                    //The glasses are in OTA mode
                }
                1, 6 ->{
                    //The glasses are in photo mode
                }
                7 -> {
                    //The glasses are communicating with AI
                }
                8 ->{
                    //The glasses are in recording mode
                }
            }
        } else {
            //Start and end execution
        }
    }
}
```

Control intelligent image recognition and report thumbnails

```
//thumbnailSize 0..6
val thumbnailSize=0x02
LargeDataHandler.getInstance().glassesControl(
        byteArrayOf(
            0x02,
            0x01,
            0x06,
            thumbnailSize.toByte(),
            thumbnailSize.toByte(),
            0x02
        )
    ) {   _,   it ->
        if (it.dataType == 1) {
```

Machine Translated by Google

```
                    if (it.errorCode == 0) {
                        when (it.workTypeIng) {
                            2 -> {
                                //The glasses are recording video
                            }
                            4 -> {
                                //The glasses are in transmission mode
                            }
                            5 -> {
                                //The glasses are in OTA mode
                            }
                            1, 6 ->{
                                //The glasses are in photo mode
                            }
                            7 -> {
                                //The glasses are communicating with AI
                            }
                            8 ->{
                                //The glasses are in recording mode
                            }
                        }
                    } else {
                        //Trigger AI to take a photo, and report the thumbnail to receive the report command
                    }
                }
            }
```

BT pairing

Machine Translated by Google

```
//BT scan
BleOperateManager.getInstance().classicBluetoothStartScan()
//BluetoothReceiver
BluetoothDevice.ACTION_FOUND -> {
val device =
intent.getParcelableExtra<BluetoothDevice>
(BluetoothDevice.EXTRA_DEVICE)
if (device != null) {
//Discover the device and call pairing when the Bluetooth address is equal to the current BLE address

BleOperateManager.getInstance().createBondBluetoothJieLi(device)
}
}
```

Glasses volume control

```
//Read volume control
LargeDataHandler.getInstance().getVolumeControl { _, response ->
if (response != null) {
// Glasses volume minimum and maximum music values current value
response.minVolumeMusic
response.maxVolumeMusic
response.currVolumeMusic
//Glass phone minimum value maximum value current value
response.minVolumeCall
response.maxVolumeCall
response.currVolumeCall
//Glass system minimum and maximum current values
response.minVolumeSystem
response.maxVolumeSystem
response.currVolumeSystem
//The current mode of the glasses
response.currVolumeType
```

Machine Translated by Google

```
                    }
                }
```

The amount of media not synced with the glasses

```
LargeDataHandler.getInstance().glassesControl(byteArrayOf(0x02, 0x04))
{ _, it ->
//it.imageCount number of images
//it.videoCount video number
//it.recordCount number of recordings
        if (it.dataType == 4) {
        val mediaCount = it.imageCount + it.videoCount + it.recordCount
        if (mediaCount > 0) {
        //How many media have not been uploaded to the glasses?
        } else {
        //No
        }
    }
}
```

Glasses WIFI connection, photo and video guide, audio recording

```
//Requires WIFI permission, please note
private val requestedPermissions = buildList {
        add(Manifest.permission.INTERNET)
        add(Manifest.permission.ACCESS_WIFI_STATE)
        add(Manifest.permission.CHANGE_WIFI_STATE)
        add(Manifest.permission.ACCESS_NETWORK_STATE)
        add(Manifest.permission.CHANGE_NETWORK_STATE)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            add(Manifest.permission.NEARBY_WIFI_DEVICES)
        } else {
            add(Manifest.permission.ACCESS_COARSE_LOCATION)
```

Machine Translated by Google

```
                add(Manifest.permission.ACCESS_FINE_LOCATION)
            }
        }.toTypedArray()

        // Initialize the folder path to store media. Please note that the APP requires permissions and creates a folder.

GlassesControl.getInstance(MyApplication.getInstance())?.initGlasses(MyA
application.getInstance().getAlbumDirFile().absolutePath)

        //Register listener

  GlassesControl.getInstance(MyApplication.getInstance())?.setWifiDownloa
dListener(object :GlassesControl.WifiFilesDownloadListener{
                //Anti-shake video end callback
                override fun eisEnd(fileName: String, filePath: String)
{
                    Log.i(TAG,"eisEnd fileName: $fileName filePath:
$filePath")
                }
                //Anti-shake video processing failed
                override fun eisError(fileName: String, sourcePath:
String, errorInfo: String) {
                    Log.i(TAG,"eisEnd fileName: $fileName filePath:
$sourcePath errorInfo: $errorInfo")
                }
                //Media upload callback, index refers to the current sequence number, total refers to the total number of media
                override fun fileCount(index: Int, total: Int) {
                    Log.i(TAG,"fileCount index: $index total: $total")
                }
                //Callback when all files are downloaded
                override fun fileDownloadComplete() {
                    Log.i(TAG,"fileDownloadComplete")
                }
                //Callback when all files fail to download
                override fun fileDownloadError(fileType: Int, errorType:
Int) {
```

Machine Translated by Google

```
                Log.i(TAG,"fileDownloadError fileType: $fileType
errorType: $errorType")
            }
            //Current file download progress callback
            override fun fileProgress(fileName: String, progress:
Int) {
                Log.i(TAG,"fileProgress fileName: $fileName
progress: $progress")
            }
            //Callback when the current file download is complete
            override fun fileWasDownloadSuccessfully(entity:
GlassAlbumEntity) {
                Log.i(TAG,"fileWasDownloadSuccessfully entity:
$entity")
            }
            //Synchronous multimedia start callback
            override fun onGlassesControlSuccess() {
                Log.i(TAG,"onGlassesControlSuccess")
            }
            //Synchronous multimedia conflict callback
            override fun onGlassesFail(errorCode: Int) {
                Log.i(TAG,"onGlassesFail errorCode: $errorCode")

            }
            //Synchronize recording files, pcm success callback
            override fun recordingToPcm(fileName: String, filePath:
String, duration: Int) {
                Log.i(TAG,"recordingToPcm fileName: $fileName
filePath: $filePath duration: $duration")
            }
              // Synchronize recording files, PCM failure callback
            override fun recordingToPcmError(fileName: String,
errorInfo: String) {
                Log.i(TAG,"recordingToPcmError fileName: $fileName
errorInfo: $errorInfo")
            }
```

Machine Translated by Google

```
//Media wifi speed callback
override fun wifiSpeed(wifiSpeed: String) {
    Log.i(TAG,"wifiSpeed wifiSpeed: $wifiSpeed")
}
// PCM data callback for glasses speaking
    override fun voiceFromGlasses(pcmData: ByteArray) {
        Log.i(TAG, ByteUtil.bytesToString(pcmData))

}

// Glasses speaking status callback, 1 start 2 end
override fun voiceFromGlassesStatus(status: Int) {
        Log.i(TAG, "voiceFromGlassesStatus$status")
}

})

//Synchronize glasses multimedia

GlassesControl.getInstance(MyApplication.getInstance)?.importAlbum()

Note that APP needs to add dependencies:
  implementation 'com.google.code.gson:gson:2.8.9'
implementation 'com.squareup.okhttp3:okhttp:4.9.3'
```